

VIXOR Full System Audit Report

Complete reverse-engineering and forensic audit of the Vixor trading platform. Covering architecture, security, database, API integrations, AI systems, data flows, and rebuild recommendations. Detailed enough for a new engineering team to rebuild the entire platform.

Z.ai Engineering Audit

Version 1.0 · June 2026

20 Sections · 24 Pages

Table of Contents

Section 1: Executive Project Overview	2
Section 2: Complete Filesystem Map	2
Section 3: Page Inventory	4
Section 4: Feature Inventory	5
Section 5: Frontend Architecture	6
Section 6: Backend Architecture	7
Section 7: Database Audit	8
Section 8: External API Inventory	10
Section 9: AI System Audit	11
Section 10: Data Flow Map	12
Section 11: User Journey Audit	13
Section 12: UX Audit	14
Section 13: Performance Audit	15
Section 14: Security Audit	16
Section 15: DevOps Audit	17
Section 16: Technical Debt Report	18
Section 17: Product Gap Analysis	18
Section 18: Core Domain Model	20
Section 19: Vixor Neural Map	21
Section 20: Master Rebuild Recommendations	22

Section 1: Executive Project Overview

Vixor is a mobile-first trading intelligence platform built on TanStack Start (React 19 + Vite 7 + Nitro serverless) deployed to Vercel with Supabase as the backend-as-a-service layer. The platform aims to serve as a unified trading operating system for retail traders, providing chart analysis, market data, AI-powered copilot assistance, portfolio tracking, daily trading routines, and social features. The application currently supports both web and Telegram Mini App contexts, with authentication via Supabase Auth (email/password) and Telegram WebApp auto-login.

The current version represents a functional MVP with real API integrations for market data (TwelveData, Binance), AI analysis (Gemini 2.5 Pro vision + local deterministic engine), and payment processing (Telegram Stars). The platform has a sophisticated local analysis engine implementing SMC/ICT methodology with 74 candlestick patterns, 20 chart formations, 8 harmonic patterns, and a 12-signal confluence scoring system. This engine produces deterministic, reproducible results without any random number generation, which is a significant architectural advantage over stochastic AI-only approaches.

However, the platform exhibits substantial technical debt across multiple dimensions: triple code duplication across src/lib/, src/shared/, and src/integrations/; approximately 80 orphaned "(1)" suffixed files in the repository root; a complete project duplicate in vixor-APP/; dead API route files; non-functional payment verification; and a critical security posture with exposed Vercel OIDC tokens, unverified Telegram webhooks, and no rate limiting or monitoring infrastructure.

Maturity Assessment

Dimension	Score	Assessment
Architecture Quality	5/10	Good domain structure but severe duplication and dead code
Product Completeness	6/10	Core features functional but shallow; no mock data in production
Scalability	4/10	Serverless-friendly but no caching strategy, 5 AI calls per consensus
Maintainability	3/10	Triple duplication, dead code, orphaned files, no CI/CD
Technical Debt	8/10	High: duplicate code, no tests, no monitoring, unverified payments

Section 2: Complete Filesystem Map

The Vixor project spans multiple directories with significant redundancy. The primary source code resides in src/ with a Domain-Driven Design (DDD) structure. However, backward-compatibility re-export layers in src/lib/, src/shared/, src/integrations/, and src/server/ create a triple-redundancy pattern where the same module exists in 2-3 locations. The following map documents every significant directory and its purpose.

Root Configuration

File	Purpose
package.json	Project manifest: TanStack Start 1.168, React 19, Vite 7, AI SDK, Supabase
vite.config.ts	Build config: Tailwind, tsconfig paths, TanStack Start, Nitro/vercel preset
vercel.json	Deployment: cron job /api/check-alerts every 5 minutes
tsconfig.json	TypeScript: ES2022, strict mode, @/* path alias
.env	Template: TWELVEDATA_API_KEY, GEMINI_API_KEY, TELEGRAM_BOT_TOKEN, etc.
.env.production	CRITICAL: Contains exposed Vercel OIDC JWT token
scripts/fix-vercel-bundle.mjs	Post-build: patches Vercel serverless bundle for NFT tracing + error handling
bunfig.toml	Bun: 24h supply-chain guard on new packages
components.json	shadcn/ui: new-york style, lucide icons

src/ Directory Structure

Directory	Purpose	Notes
src/routes/	File-based routing (TanStack Router)	17 page routes + 4 API routes + 4 dead API routes
src/domains/	Business domain layer (DDD)	9 domains: analysis, market, trading, copilot, user, watchlist, notes, trades, daily-loop
src/components/vixor/	App-specific components	AppShell, OnboardingModal, TradingViewChart, AlertsList, etc.
src/components/ui/	shadcn/ui library	47 Radix-based UI components
src/lib/	Backward-compatible re-exports	DUPLICATE of src/shared/ + src/domains/
src/shared/	Shared utilities (canonical)	Supabase clients, hooks, i18n, cache
src/integrations/supabase/	Supabase integration (legacy)	DUPLICATE re-export of src/shared/supabase/
src/server/	Server modules (legacy)	Re-exports from src/domains/*/server/
src/hooks/	Custom hooks (legacy)	Re-exports from src/shared/hooks/

Duplicate / Dead Code Directories

The following directories and files represent redundancy or dead code that inflates repository size and increases maintenance burden:

- **vixor-APP/** - Complete project clone with its own .git/, node_modules/, src/, supabase/. Missing 4 recent migrations. Should be removed or git-ignored.
- **~80 root "(1)" files** - File sync conflict artifacts (button (1).tsx, card (1).tsx, etc.). Not imported, dead weight.
- **3 stray SQL files at root** - Migration SQL files that belong in supabase/migrations/.
- **src/routes/api/-check-alerts.ts** and 3 other "-prefixed" API files - Dead workaround copies.
- **src/types/api-routes.d.ts** - Declares non-existent createAPIFileRoute module.
- **src/lib/vixor-mock.ts** - Mock data module (deprecated but still present).
- **src/lib/analysis/** - Complete duplicate of src/domains/analysis/engine/.

Section 3: Page Inventory

The Vixor platform implements 17 page routes under an authenticated layout guard, plus a public authentication page and 4 server-side API endpoints. Every production route uses real data from Supabase or external APIs. Zero mock data exists in production page components. The only mock-related file is src/lib/vixor-mock.ts which is deprecated but still present in the codebase.

Page	URL	Data Source	Status	UX Quality	Production Ready
Dashboard	/	Real (Supabase + APIs)	Working	7/10	Partial
Auth	/auth	Real (Supabase Auth)	Working	8/10	Yes
Analyze	/analyze	Real (AI + Supabase)	Working	7/10	Yes
Analysis Result	/analysis/\$id	Real (Supabase)	Working	8/10	Yes
Charts	/charts	Real (TradingView + APIs)	Working	7/10	Yes
Copilot	/copilot	Real (AI + Supabase)	Working	7/10	Partial
Daily Loop	/daily-loop	Real (Supabase)	Working	6/10	Partial
Discover	/discover	Real (Supabase + APIs)	Working	7/10	Yes
Journal	/journal	Real (Supabase)	Working	6/10	Partial
Portfolio	/portfolio	Real (Supabase)	Working	7/10	Yes
Signals	/signals	Real (Supabase)	Working	7/10	Yes

Trade Desk	/trade-desk	Real (Local calc + DB)	Working	6/10	Partial
Notifications	/notifications	Real (Supabase)	Working	5/10	Partial
Profile	/profile	Real (Supabase)	Working	6/10	Yes
Premium	/premium	Real (Supabase + TG)	Working	5/10	No
Referral	/referral	Real (Supabase)	Working	5/10	No
Settings	/settings	LOCAL (not persisted)	Broken	3/10	No

Settings Page Critical Issue: Dark mode, haptics, sound, and notification toggles are not persisted to the database. They reset on every page reload, making the settings page effectively non-functional for any user who expects their preferences to persist across sessions.

Section 4: Feature Inventory

The feature inventory below catalogs every implemented feature in the Vixor platform, classifying each by its connection status to backend services and identifying whether the functionality is real, mock, or missing. The platform has made significant progress in replacing mock data with real Supabase-backed implementations, but several critical gaps remain, particularly in payment verification and alert triggering.

Feature	Backend	Database	API	Real ?	Classification
Chart Analysis (AI)	Yes	Yes	Yes	Yes	Production Ready
Local Analysis Engine	Yes	Yes	N/A	Yes	Production Ready
Market Prices	Yes	No	Yes	Yes	Production Ready
Price Alerts	Partial	Yes	Yes	Broken	CRITICAL: Cron returns stub "ok"
Daily Signals	Partial	Yes	Yes	Broken	CRITICAL: Cron returns stub "ok"
AI Copilot (Multi-Agent)	Yes	Yes	Yes	Yes	Production Ready
Consensus Mode	Yes	Yes	Yes	Yes	Expensive (5 AI calls/msg)
Watchlist CRUD	Yes	Yes	Yes	Yes	Production Ready

Trading Notes	Yes	Yes	Yes	Yes	Production Ready
Trade Tracking	Yes	Yes	Yes	Yes	Production Ready
Portfolio Stats	Yes	Yes	Yes	Yes	Production Ready
Daily Loop Routine	Yes	Yes	Yes	Yes	Production Ready
Economic Calendar	Yes	No	Yes	Yes	Limited (this week only)
Market News	Yes	No	Yes	Yes	Production Ready
Telegram Auth	Yes	Yes	Yes	Yes	Production Ready
Premium Subscription	Yes	Yes	No	FAKE	No payment verification
Point Pack Purchase	Yes	Yes	No	FAKE	No payment verification
Telegram Stars Payment	Partial	Yes	Yes	Broken	Webhook unverified
Referral System	Yes	Yes	Yes	Yes	Production Ready
Notifications (In-App)	Partial	Yes	Yes	Broken	Alerts never trigger (cron stub)
Notifications (Telegram)	Partial	Yes	Yes	Broken	Same cron issue
Settings Persistence	No	No	No	FAKE	Client-side only, resets on reload
i18n (EN/AR)	Yes	No	N/A	Yes	Production Ready

Section 5: Frontend Architecture

The frontend is built on TanStack Router with file-based routing, React 19, and a component library based on shadcn/ui (new-york variant) with 47 Radix UI primitives. The layout system uses an authenticated guard pattern where all protected routes are children of `_authenticated/route.tsx`, which checks Supabase session validity in `beforeLoad` and redirects unauthenticated users to `/auth`. The root layout (`__root.tsx`) provides `QueryClientProvider`, `I18nProvider`, `GlobalErrorBoundary`, and the `AppShell` component which contains a sticky glass header and bottom navigation bar.

State Management

State management relies primarily on TanStack React Query with conservative defaults: `refetchOnWindowFocus`, `refetchOnReconnect`, and `refetchOnMount` are all disabled; `staleTime` is 30 seconds; `retry` is limited to 1 attempt; and `structuralSharing` is enabled to reuse object references and prevent unnecessary re-renders. These settings were deliberately chosen to prevent the React #310 render loop issue that previously plagued the application. The trade-off is that users may see slightly stale data on navigation, but this is preferable to infinite re-render loops.

Additionally, the application uses Jotai atoms for minimal local state, and `localStorage` for persistence of the onboarding flag (`vixor-onboarded`) and settings state (which is not synced to the database). Auth state is managed through a single listener in `__root.tsx` with 500ms debounce and 2-second deduplication window to prevent cascading invalidations.

Identified Issues

- **Triple code duplication:** `src/lib/`, `src/shared/`, and `src/integrations/` contain overlapping Supabase client, hooks, `i18n`, and utility files. The canonical implementations are in `src/shared/`, while `src/lib/` and `src/integrations/` are backward-compatibility re-exports.
- **Dual analysis engine copies:** `src/domains/analysis/engine/` and `src/lib/analysis/` are duplicates, creating maintenance risk if only one is updated.
- **~80 root-level "(1)" files:** File sync conflict artifacts that pollute the repository. These are not imported by any module.
- **Settings not persisted:** All settings (dark mode, language, notifications) are client-side only and reset on reload. This breaks the user experience for returning users.
- **`refetchOnMount`: false:** Users navigating back to a page will see stale data until the 30s `staleTime` expires. This is a deliberate trade-off but can confuse users.
- **`defaultPreload`: false:** Route preloading is disabled to prevent cascading re-renders, making navigation feel slower than optimal.

Section 6: Backend Architecture

The backend architecture uses TanStack Start server functions (`createServerFn`) as the primary RPC mechanism, with H3 event handlers for API routes. All server functions go through a two-layer middleware chain: (1) client-side `auth-attacher.ts` attaches the Supabase bearer token to every server function call, and (2) server-side `auth-middleware.ts` validates the token and provides an authenticated Supabase client with user context. This pattern ensures that every server function has access to the current user identity without manual token management.

Server Functions Data Flow

The standard data flow for any authenticated operation is: User Action (React component) calls `useStableServerFn()` wrapper, which invokes a `createServerFn` function. The client middleware attaches the Bearer token from the current Supabase session. The server middleware validates the token via `supabase.auth.getUser()`, extracts `userId` and `claims`, and provides these in the middleware context. The handler function uses the authenticated Supabase client or the admin client (`supabaseAdmin` with service role key) to perform database operations. Results are returned to the client and cached by React Query.

API Endpoints (Server-Side)

Endpoint	Method	Auth	Purpose	Functional?
/api/check-alerts	GET/POST	CRON_SECRET (optional)	Check all price alerts against live prices	NO - Returns stub "ok"
/api/generate-signals	GET/POST	CRON_SECRET (optional)	Generate daily signals for 6 pairs x 2 timeframes	NO - Returns stub "ok"
/api/telegram-webhook	POST	None	Process Telegram payment webhooks	NO - Returns stub "ok"
/api/migrate	GET/POST	None	Check migration status / get pending SQL	Yes

CRITICAL: The `fix-vercel-bundle.mjs` script intercepts all `/api/*` routes and returns static "ok" JSON responses WITHOUT executing any actual logic. This means price alerts are never checked, signals are never generated, and Telegram payment webhooks are never processed. This is the single most critical bug in the current deployment.

Background Jobs

The only background job is a Vercel cron that calls `/api/check-alerts` every 5 minutes. However, as noted above, this endpoint is currently non-functional due to the `fix-vercel-bundle.mjs` API route interception. There is no queue system, no job scheduler beyond Vercel cron, and no retry mechanism for failed operations. The `generate-signals` endpoint is intended to be called manually or via a separate cron schedule that is not configured.

Section 7: Database Audit

The database is hosted on Supabase (PostgreSQL 15+) with Row Level Security (RLS) enabled on all user-scoped tables. The schema consists of 19 tables, 3 enums, and 8 database functions/triggers. Migrations are managed through SQL files in `supabase/migrations/` and a runtime migration checker in `src/shared/migrate.server.ts`. The project does not use Prisma despite initial plans; all database access is through the Supabase JS client.

Complete Table Inventory

Table	RLS	Key Columns	Purpose
profiles	Yes	id, telegram_id, referral_code, streak_days, xp fields	User profiles with Telegram linking
points_balances	Yes	user_id, balance, lifetime_earned	Virtual currency balances
points_transactions	Yes	user_id, delta, reason, metadata	Points ledger with enum reasons
point_packs	Yes	id (TEXT PK), points, price_cents	Purchasable point bundles
premium_plans	Yes	id (TEXT PK), price_cents, interval, features	Subscription tier definitions
premium_subscriptions	Yes	user_id, plan_id, status, period_end	Active premium subscriptions
analyses	Yes	pair, timeframe, recommendation, confidence, entry/SL/TP	AI analysis results with SMC data
notifications	Yes	user_id, title, body, type, read_at	In-app notification queue
watchlists	Yes	user_id, name, is_default	User watchlist containers
watchlist_items	Yes	watchlist_id, pair, category	Assets within watchlists
price_alerts	Yes	user_id, pair, condition, target_price, status	User-configured price alerts
daily_signals	No	pair, timeframe, recommendation, signal_date	System-generated trading signals
user_strategies	Yes	user_id, name, pairs[], trading_style	User trading strategy config
trading_notes	Yes	user_id, pair, title, content, tags[], mood	Journal notes with mood tracking
trades	Yes	user_id, pair, direction, entry/exit, pnl (GENERATED)	Trade log with computed P&L;
copilot_conversations	Yes	user_id, title, agent_id	AI chat conversation threads
copilot_messages	Yes	conversation_id, role, content, agent_id	Individual chat messages
daily_loops	Yes	user_id, date, morning_prep, sessions, eod_review	Daily trading routine data
user_streaks	Yes	user_id, current_streak, longest_streak	User engagement streaks

Database Issues

- daily_signals has NO RLS:** Any authenticated user can read all signals. This is intentional (public catalog) but means any user can query all signal data, including future signals before they are meant to

be visible.

- **Schema drift:** TypeScript types in `src/shared/supabase/types.ts` include columns (`telegram_photo_url`, `telegram_username`, `xp`) that do not exist in any migration SQL file. These were likely added manually or in untracked migrations.
- **Enum mismatch:** The `points_reason` enum in TypeScript includes `telegram_stars_purchase` but the base SQL enum does not. This was likely added via `ALTER TYPE` outside tracked migrations.
- **Duplicate updated_at functions:** `set_updated_at()` and `update_updated_at()` do the same thing with different names. The former is from the base schema, the latter from a later migration.
- **opportunity_id column:** Added to analyses in Migration 2 but references no foreign key and appears unused.
- **trades table uses GENERATED STORED columns:** `pnl`, `pnl_pips`, and `r_multiple` are computed columns. While this ensures consistency, it means these values cannot be overridden even when needed for manual corrections.
- **No migration versioning:** There is no mechanism to track which migrations have been applied. The runtime checker only tests for table existence, not schema correctness.

Section 8: External API Inventory

The Vixor platform integrates with 5 external API providers for market data, AI analysis, and payment processing. Each integration has its own fallback strategy and error handling approach. The most critical finding is that all API keys are passed as URL query parameters (TwelveData, Finnhub, Telegram), which exposes them in server logs, proxy caches, and network monitoring tools.

Provider	Endpoints	Purpose	Active	Key Exposure	Fallback
Binance	<code>/api/v3/ticker/24hr</code> , <code>/api/v3/klines</code>	Crypto prices + OHLCV	Yes	No key needed	TwelveData
TwelveData	<code>exchange_rate</code> , <code>time_series</code> , <code>quote</code> , <code>ETFs</code> , <code>fundamentals</code>	Forex/commodity prices + ETFs	Yes	API key in URL	exchangerate-api
ExchangeRate-API	<code>/v4/latest/{base}</code>	Forex rates (free)	Yes	No key needed	Cache, then null
Finnhub	<code>/v1/news</code> , <code>/v1/calendar/economic</code>	News + economic calendar	Yes	API key in URL	Faireconomy (calendar)
Faireconomy Media	<code>/ff_calendar_thisweek.json</code>	Economic calendar (free)	Yes	No key needed	Finnhub
Google Gemini 2.5	Vision API via Vercel AI SDK	Chart image analysis (fallback)	Yes	Server-side key	Local engine

Lovable AI Gateway	ai.gateway.lovable.dev/v1	Copilot AI (primary)	Yes	Server-side key	Direct Gemini
Telegram Bot API	sendMessage, createInvoiceLink	Notifications + payments	Yes	Token in URL	None

API Key Security Issues

The most concerning pattern is the exposure of API keys in URL query parameters. Both TwelveData and Finnhub require API keys as query parameters by design, but this means every request URL containing the key is logged by Vercel, any CDN, and any network monitoring tool. The Finnhub key is used in run-analysis.ts for fetching news context, and the TwelveData key is used in both price-fetcher.ts and twelvedata.ts for market data. The Telegram bot token is similarly exposed in alert-checker.ts when sending notification messages. These keys should be rotated and the integrations should be refactored to use header-based authentication where possible, or at minimum, the server-side logging should be configured to redact these parameters.

Section 9: AI System Audit

The Vixor AI system operates on a two-tier architecture: (1) a local deterministic analysis engine that runs first and produces reliable, reproducible results without any API calls, and (2) a Gemini 2.5 Pro vision AI fallback that activates only when local confidence is below 60% and the API key is available. This architecture is a significant strength, as it ensures the platform can function without external AI dependencies and produces consistent results for the same input data.

Local Analysis Engine

The local engine implements a comprehensive SMC/ICT (Smart Money Concepts / Inner Circle Trader) methodology through an 8-stage pipeline: OHLCV data acquisition (returns WAIT if less than 20 bars, never fabricates data), market structure analysis (swing points, trend direction), SMC concepts (Order Blocks, Fair Value Gaps, Liquidity Zones, Support/Resistance), BOS/CHoCH detection, pattern detection (74 candlestick + 20 chart formations + 8 harmonic patterns), technical indicators (RSI, MACD, Bollinger, EMA, ADX, Stochastic), risk-reward calculation, and confluence scoring. The confluence system requires a minimum of 3 aligned signals from 12 independent indicators to produce a BUY or SELL recommendation; otherwise, it defaults to WAIT. This is an excellent design that prevents low-confidence recommendations.

AI Copilot (Multi-Agent System)

The copilot system defines 4 specialized agents: Market Analyst (default), Risk Manager, News Analyst, and Strategy Builder. Each agent has a unique system prompt parameterized with user context (profile, recent analyses, signals, alerts, watchlist, market prices, economic events). Auto-selection routes messages based

on keyword matching, with Arabic keyword support. The system supports two modes: single-agent (routes to the most relevant agent) and consensus (runs all 4 agents in parallel with shorter responses, then synthesizes a unified answer via a 5th AI call).

AI System Issues

- **170 lines of dead fake-news code:** engine.ts contains a generateNewsContext() function with hardcoded fake news headlines that is never called. The result explicitly sets newsImpact = undefined. This dead code is misleading and should be removed.
- **Dead calculateConfidence() function:** Defined but never called. Confidence is calculated inline using confluence.score.
- **Dynamic imports on every AI call:** agent-orchestrator.ts uses await import("ai") and await import("@ai-sdk/google") on every single call, with no caching. This is a performance issue and could cause module duplication in serverless environments.
- **Consensus mode is expensive:** Each user message in consensus mode triggers 5 AI calls (4 agents + 1 synthesis). At scale with multiple concurrent users, this could exhaust API quotas rapidly.
- **No rate limiting on AI calls:** Any authenticated user can make unlimited copilot requests, including consensus requests. There are no cost controls or rate limits.
- **No streaming:** All AI responses are collected fully before returning to the client. Users wait for the entire response before seeing any output, which creates a poor chat experience.
- **Hallucination risk:** While the local engine is deterministic and hallucination-free, the Gemini fallback and copilot agents are susceptible to hallucination. The copilot system prompts are well-structured but do not include explicit anti-hallucination instructions or fact-checking protocols.

Section 10: Data Flow Map

This section maps how data moves across the Vixor platform, documenting implemented flows and identifying missing connections. Understanding these data flows is critical for identifying where the platform fails to deliver on its promise of a "unified trading operating system."

Implemented Data Flows

- **Market Data Flow:** Binance/TwelveData APIs fetchPrice() in-memory cache (Upstash or 5-min TTL) React Query cache (30s stale) UI components. Prices are never stored in the database; they are always fetched fresh from external APIs.
- **Analysis Flow:** User uploads chart image createAnalysis server function image bytes + OHLCV data local engine (deterministic) OR Gemini vision API (fallback) analysis result saved to analyses table React Query invalidation UI update. Analysis results are persisted in Supabase.

- **Copilot Flow:** User sends message askCopilot server function agent orchestrator Lovable AI Gateway OR Gemini createMessage Lovable AI Gateway agent response save to copilot_messages table UI streaming response. Conversation history is persisted.
- **Watchlist Flow:** User adds/removes pair createWatchlist/addToWatchlist server functions Supabase watchlists/watchlist_items tables React Query invalidation UI update. Fully functional CRUD.
- **Trade Flow:** User creates trade createTrade server function Supabase trades table (GENERATED columns compute pnl/pnl_r_multiple) React Query invalidation portfolio stats + equity curve update. Fully functional.
- **Alert Flow (BROKEN):** User creates alert Supabase price_alerts table (status: active) Vercel cron /api/check-alerts (every 5 min) STUB: returns "ok" without checking alerts never triggered, notification never sent, user never informed.

Missing Data Flows

- **Analysis to Alert:** When an analysis produces a BUY/SELL signal with entry/SL/TP, there is no automated flow to create a price alert from those levels. Users must manually navigate to the charts page and create an alert.
- **Signal to Portfolio:** Daily signals do not feed into the portfolio tracker. A user cannot mark a signal as "taken" and track its outcome against their portfolio.
- **Copilot to Journal:** Copilot conversations and agent recommendations are not linked to the trading journal. Insights from AI discussions are lost when the conversation is deleted.
- **News to Analysis:** Market news is displayed on the dashboard but is not incorporated into analysis results (the generateNewsContext function exists but is deliberately not called in the local engine).
- **Economic Calendar to Daily Loop:** Economic calendar events are not surfaced in the Daily Loop morning prep phase, where they would be most relevant for pre-market planning.
- **Alert Trigger to Copilot:** When an alert is triggered, there is no flow to notify the copilot or provide context for follow-up analysis. The alert system and the AI system operate in complete isolation.

Section 11: User Journey Audit

Simulating the complete user journey from first visit through daily trading operations reveals multiple breaks in the experience. While the core analysis and market data flows work well, the platform fails to connect these capabilities into a cohesive daily workflow that would keep traders engaged and returning.

Journey Breakdown

- **New User Onboarding:** First visit triggers OnboardingModal (localStorage flag). User sees a brief introduction but no guided tour of features. After closing the modal, they land on the dashboard with no clear next step. The onboarding does not collect trading preferences, risk tolerance, or watched

instruments. MISSING: Personalization step, strategy setup, watchlist seeding.

- **First Analysis:** User must navigate to /analyze, upload a chart image, and wait for processing. There is no sample analysis or tutorial analysis to demonstrate the feature before the user spends their points. The minimum 10-point cost for free users is not clearly communicated before upload. MISSING: Sample/demo analysis, cost transparency.
- **Alert Creation:** User creates a price alert on the charts page. The alert is saved to the database with status "active." However, because the /api/check-alerts cron endpoint returns a stub "ok" response, the alert will NEVER be triggered. The user has no way to know this. DEAD END: Alerts are created but never checked.
- **Daily Routine:** User opens /daily-loop and completes morning prep. This data is saved to the database. However, the morning prep does not pre-populate with relevant data (economic calendar events, overnight price changes, watchlist performance). The daily loop is a form, not an intelligent assistant. SHALLOW: Data entry without intelligence.
- **Payment:** User navigates to /premium and clicks "Subscribe." The subscribePremium server function creates a premium subscription record in the database with status "active" without any payment verification. Similarly, point pack purchases credit points without payment. FAKE: Premium is granted freely to anyone who calls the endpoint.
- **Settings:** User changes dark mode, language, and notification preferences. All settings are stored in React state only and reset on page reload. BROKEN: Settings do not persist.

Section 12: UX Audit

The UX audit reveals a platform with strong visual design (Bloomberg Professional dark theme with trading-specific semantic colors) but several navigational and discoverability issues. The bottom navigation bar provides access to 5 core sections (Home, Discover, Analyze, Copilot, Portfolio), but several important features (Journal, Daily Loop, Signals, Trade Desk) are only accessible through the dashboard or deep links, making them hard to discover for new users.

Navigation Issues

- **Hidden pages:** Journal, Daily Loop, Signals, Trade Desk, Notifications, Profile, Settings, Premium, and Referral are not in the bottom navigation. They are only accessible through dashboard cards or the profile menu. This creates a discoverability problem where users may not know these features exist.
- **Bottom nav overload risk:** Adding more tabs to the bottom navigation would make it cramped on mobile devices. The current 5-tab design is at the practical limit for mobile UX.
- **No search:** There is no global search functionality. Users cannot search for instruments, analyses, notes, or conversations across the platform.
- **Deep-link fragility:** Several pages depend on search params (charts page with ?screenshot=base64 and ?pair=, analysis page with ?id=) but have no graceful fallback when these params are missing or

invalid.

Consistency Issues

- **Chart analysis vs. Copilot:** The chart analysis page provides structured trading recommendations with entry/SL/TP, while the copilot provides conversational advice. There is no way to use a copilot recommendation as a structured trade setup, or to ask the copilot about a specific analysis result.
- **Watchlist vs. Signals:** Watchlist items do not show associated signals, and signals do not reference watchlist membership. Two separate views of the same instruments with no cross-linking.
- **Portfolio vs. Trade Desk:** The portfolio page shows trade history and statistics, while the trade desk provides a risk calculator. These are conceptually related but physically separate pages with no navigation between them.

Section 13: Performance Audit

The performance audit identifies several bottlenecks that affect user experience, particularly around data fetching patterns, bundle size, and serverless cold starts. The most significant performance concern is the consensus mode in the copilot, which makes 5 sequential AI API calls before returning any output to the user.

Identified Bottlenecks

Issue	Severity	Impact	Location
Consensus mode: 5 AI calls per message	High	3-10s delay, high API cost	agent-orchestrator.ts
Sequential price fetch in alert checker	Medium	$N * 2-5s$ for N pairs	alert-checker.ts
Dynamic imports on every AI call	Medium	50-200ms overhead per call	agent-orchestrator.ts
No request deduplication for prices	Medium	Duplicate API calls for same pair	price-fetcher.ts
refetchOnMount: false	Low	Stale data shown on navigation	router.tsx
No route preloading	Low	Slower navigation feel	router.tsx
47 shadcn/ui components in bundle	Low	Larger initial JS bundle	components/ui/
Triple code duplication	Low	Larger source + build size	lib/, shared/, integrations/

The Vercel serverless deployment model introduces cold start latency, which is compounded by the fix-vercel-bundle.mjs post-build patching that modifies the generated serverless bundle. Each cold start must load the full Nitro server runtime, TanStack Start framework, and all imported modules. The lazy-initialization pattern used for Supabase clients helps mitigate this by deferring the createClient call to

first access, but the overall cold start time is still significant for a trading platform where users expect sub-second responses.

Section 14: Security Audit

The security audit reveals a platform with critical vulnerabilities across authentication, payment processing, API exposure, and operational security. While the Supabase RLS implementation is solid (all user-scoped tables have proper policies), the application layer above RLS has several exploitable gaps that could allow unauthorized access, free premium features, and data exposure.

Critical Vulnerabilities

- **Exposed Vercel OIDC Token in .env.production:** A full JWT containing project ID (vixor-app), owner ID, team name (goldarenax-7278s-projects), and user ID is committed to the repository. Even though expired, it reveals internal Vercel org structure and identifiers. This token must be rotated and removed from git history.
- **No Payment Verification:** `purchasePack()` and `subscribePremium()` grant points and premium status without any payment verification. Anyone can call these server functions and receive free benefits. The `createStarsInvoice()` function creates an invoice link but never verifies that payment was completed.
- **Telegram Webhook Unverified:** The `/api/telegram-webhook` endpoint processes payment events with zero signature verification. An attacker could forge webhook payloads to credit themselves points by crafting a successful `_payment` event with arbitrary `user_id` and `pack_id`.
- **Debug Error Handler in Production:** `fix-vercel-bundle.mjs` injects a `__vixor_debug__` function that renders full error messages, stack traces, and environment variable status (whether `SUPABASE_URL` and `ANON_KEY` are set or missing) as HTML on every unhandled error.
- **CSRF Protection Disabled:** The CSRF middleware warning is explicitly suppressed in `vite.config.ts` with `disableCsrfMiddlewareWarning: true`. No CSRF protection is implemented.
- **Admin Client Falls Back to Anon Key:** `supabaseAdmin` in `client.server.ts` falls back through `SUPABASE_SERVICE_ROLE_KEY` then `SUPABASE_PUBLISHABLE_KEY` then `SUPABASE_ANON_KEY`. If the service role key is not configured, operations that should bypass RLS silently use the anon key with RLS, potentially causing confusing failures or exposing only public data.
- **Cron Endpoint Auth Optional:** `/api/check-alerts` validates `CRON_SECRET` only if it is set. If not configured, anyone can trigger alert checks, which call external APIs (TwelveData, Telegram).
- **Wildcard CORS:** `fix-vercel-bundle.mjs` adds `Access-Control-Allow-Origin: *` on all API routes, allowing any origin to call server endpoints.
- **Deterministic Telegram Passwords:** Telegram users get deterministic passwords derived from HMAC-SHA256 of bot token + user ID. If the bot token leaks, all Telegram users passwords are compromised. The email format (tg-{id}@vixor.app) is also predictable.

Security Score

Category	Score	Status
Authentication	7/10	Good: Supabase Auth + Telegram HMAC verification
Authorization (RLS)	8/10	Good: All user-scoped tables have RLS policies
Authorization (App)	3/10	Poor: Server functions lack permission checks
Payment Security	1/10	Critical: No verification, free premium for all
API Key Exposure	2/10	Critical: Keys in URL query parameters
CSRF Protection	1/10	None: Warning explicitly suppressed
Rate Limiting	0/10	None: No rate limiting anywhere
Monitoring/Alerting	0/10	None: No Sentry, Datadog, or any observability
Content Security	2/10	No CSP, no security headers

Section 15: DevOps Audit

The DevOps audit reveals a platform with minimal operational infrastructure. The deployment relies entirely on Vercel serverless functions with no CI/CD pipeline, no containerization, no infrastructure as code, and no monitoring or alerting. The only automation is the Vercel cron job for alert checking (which is currently non-functional).

Area	Status	Risk Level	Notes
CI/CD Pipeline	None	Critical	No automated testing, linting, or deployment gates
Containerization	None	Medium	No Dockerfile for reproducible builds
Infrastructure as Code	None	Medium	No Terraform/Pulumi for Supabase/Vercel
Monitoring/Alerting	None	Critical	No error tracking, APM, or uptime monitoring
Rate Limiting	None	High	API abuse vector, unlimited AI calls
CORS Policy	Wildcard *	High	Cross-origin attack vector
Security Headers	None	High	No CSP, X-Frame-Options, HSTS
Dependency Audit	None in CI	Medium	No npm audit in automated pipeline
Supply Chain	bunfig.toml 24h guard	Low	Good: protects against typosquatting
Import Protection	Vite config	Low	Good: server code protected from client

Auth Middleware	Bearer token	Low	Good: token validation on every request
-----------------	--------------	-----	---

The deployment workflow is entirely manual: developers push to git, Vercel auto-deploys, and the post-build script patches the serverless bundle. There are no deployment previews, no staging environment, no automated rollbacks, and no health checks. The fix-vercel-bundle.mjs script is a fragile patch that relies on string matching in minified code and could break with any Vite or Nitro version change.

Section 16: Technical Debt Report

Technical debt in the Vixor platform spans multiple categories: dead code, duplicate systems, abandoned features, temporary implementations, and dangerous architecture decisions. The following table ranks each debt item by severity and estimated remediation effort.

Debt Item	Category	Severity	Effort	Impact
fix-vercel-bundle.mjs API stubs	Dangerous	Critical	Medium	Alerts, signals, payments all broken
No payment verification	Dangerous	Critical	Medium	Free premium for all callers
Triple code duplication (lib/shared/integrations)	Duplicate	High	High	3x maintenance burden
vixor-APP/ full project clone	Duplicate	High	Low	2x repo size, stale migrations
~80 root "(1)" files	Dead Code	Medium	Low	Repository pollution
Dead API routes (-check-alerts, etc.)	Dead Code	Medium	Low	Confusion, build bloat
170 lines of dead fake-news code	Dead Code	Medium	Low	Misleading codebase
Dead calculateConfidence() function	Dead Code	Low	Low	Minor confusion
Settings not persisted	Temporary	High	Medium	Broken user experience
Dynamic imports on every AI call	Temporary	Medium	Low	Performance overhead
Schema drift (TS types vs. SQL)	Dangerous	Medium	Medium	Runtime type errors possible
Duplicate updated_at functions	Duplicate	Low	Low	Minor confusion
src/types/api-routes.d.ts dead declaration	Dead Code	Low	Low	Build noise
No test suite	Missing	High	High	No regression protection

Section 17: Product Gap Analysis

Comparing the current Vixor implementation against the vision of a unified trading operating system reveals significant gaps across three dimensions: missing features that are essential for the vision, incomplete

features that exist but lack depth, and unnecessary complexity that adds maintenance burden without user value.

What is Missing

- **Real-time price streaming:** All price data is fetched via REST polling (60s intervals on dashboard). A trading platform needs WebSocket-based real-time price feeds for live charts and instant alert triggering.
- **Push notifications:** Alert notifications rely on the broken cron endpoint. Even if fixed, 5-minute polling intervals are insufficient for trading alerts. Push notifications (browser push, Telegram, or native mobile) are essential.
- **Backtesting:** There is no way to test a strategy against historical data. The analysis engine could theoretically support backtesting, but no UI or API exists for it.
- **Social/Community features:** Despite being listed in the product vision, there are no social features: no shared watchlists, no signal sharing, no community discussions, no leaderboards.
- **Multi-account / Broker integration:** Portfolio tracking is manual (users enter trades). There is no broker API integration for automatic trade import or portfolio sync.
- **Advanced order types:** The alert system supports price alerts only. There are no conditional orders, trailing stops, OCO orders, or bracket orders.

What is Incomplete

- **Economic Calendar:** Only shows current week events (Faireconomy API limitation). No historical events, no impact tracking, no integration with daily loop or analysis.
- **Copilot:** Functional but shallow. No streaming, no tool use (cannot create alerts, place trades, or fetch data), no conversation memory beyond the current session, no proactive insights.
- **Portfolio Intelligence:** Shows basic stats (win rate, P&L, equity curve) but lacks advanced analytics (drawdown analysis, Sharpe ratio, correlation matrix, sector exposure).
- **Daily Loop:** A form-based data entry tool rather than an intelligent assistant. Does not pre-populate with relevant market data or learn from past routines.

What is Overbuilt

- **47 shadcn/ui components:** Many registered UI components (carousel, command, context-menu, menubar, etc.) are never used in any page. These add to the bundle size without benefit.
- **ETF/fundamentals integration:** TwelveData ETF directory, performance, summary, cash flow, earnings estimates, and growth estimates are implemented in twelvedata.ts but not used by any page or feature.
- **Dual AI provider (Lovable + Gemini):** The Lovable AI Gateway adds a dependency and complexity. Direct Gemini access would be simpler and sufficient.

Section 18: Core Domain Model

The core domain model of a trading operating system consists of the following entities and their relationships. This section documents the current implementation status of each entity and identifies missing relationships that prevent Vixor from operating as a unified system.

Entity Implementation Status

Entity	Implemented	Database	Relationships	Gaps
User	Yes	profiles + points_balances	Premium, Referrals	Settings not persisted
Asset (Pair)	Partial	No table (hardcoded)	WatchlistItem, Alert, Trade	No asset master table
Trade	Yes	trades	User, Asset	No link to Analysis or Signal
Analysis	Yes	analyses	User, Asset	No link to Alert, Trade, Journal
News	Partial	No table (API only)	None	Not linked to Asset or Analysis
Journal/Note	Yes	trading_notes	User, Asset	No link to Trade or Analysis
Alert	Yes (broken)	price_alerts	User, Asset	No link to Analysis, never triggers
Portfolio	Derived	trades (computed)	User	No asset allocation, no benchmarks
Wallet/Points	Yes	points_balances + transactions	User	No real payment verification
Community	No	None	None	Not implemented
Strategy	Partial	user_strategies	User	Not linked to signals or trades
Signal	Yes	daily_signals	None	Not linked to Asset, Trade, or Alert

Missing Relationships (The Neural Gap)

The most critical gap in the domain model is the lack of cross-entity relationships. Currently, each entity operates in isolation: analyses do not connect to alerts, signals do not connect to trades, and the copilot does not connect to any other entity. A true trading operating system requires these entities to form a neural network where every action creates context for subsequent actions. The following relationships are missing:

- **Analysis to Alert:** Analysis produces entry/SL/TP levels. These should auto-generate price alerts.

- **Signal to Trade:** User should be able to "take" a signal and create a trade from it.
- **Trade to Journal:** Closing a trade should prompt a journal entry with pre-populated trade data.
- **Copilot to Any Entity:** Copilot should be able to read and write alerts, trades, notes, and analyses.
- **News to Analysis:** Relevant news should be incorporated into analysis results.
- **Economic Event to Daily Loop:** Upcoming high-impact events should appear in morning prep.
- **Asset Master Table:** All pair references (in watchlists, alerts, trades, analyses, signals) should reference a central asset registry with metadata (pip size, trading hours, category).

Section 19: Vixor Neural Map

The Vixor Neural Map represents how platform entities should be connected to create a living, breathing trading operating system. Currently, the platform operates as a collection of isolated tools rather than an integrated neural network. This section documents both the current state and the target state.

Current Connections (Isolated Islands)

Source	Target	Connection	Status
Dashboard	Market Prices	Fetch + display	Working
Dashboard	Daily Signals	Fetch + display	Working
Dashboard	Watchlist	Fetch + display	Working
Charts	Price Alerts	Create alert from chart	Working (but never triggers)
Analyze	Analysis Result	Create + save	Working
Copilot	Conversations	Save messages	Working
Trade Desk	Trades	Create trade	Working
Portfolio	Trade Stats	Compute from trades	Working

Target Connections (Neural Network)

Source	Target	Connection	Priority
Analysis	Price Alert	Auto-create alerts from entry/SL/TP levels	P0
Signal	Trade	"Take signal" creates a trade	P0
Trade (closed)	Journal Note	Auto-prompt journal entry with trade data	P1
Copilot	All Entities	Read/write alerts, trades, notes, analyses	P1

Economic Calendar	Daily Loop	Pre-populate morning prep with events	P1
News	Analysis	Incorporate relevant news into analysis context	P2
Alert (triggered)	Copilot	Provide alert context for follow-up analysis	P2
Signal	Price Alert	Create alert from signal levels	P2
Asset Master	All Entities	Central registry for pair metadata	P2
Community	Signals/Analysis	Share and discuss trading ideas	P3
Backtest Engine	Strategy	Test strategies against historical data	P3
Broker API	Portfolio	Auto-import trades and positions	P3

Section 20: Master Rebuild Recommendations

This final section provides a phased roadmap for transforming Vixor from its current state into the unified trading operating system it aims to be. The recommendations are organized by what should remain untouched, what should be refactored, what should be rebuilt, what should be removed, and what should be postponed. Each phase is designed to be independently valuable, so the platform improves incrementally rather than requiring a complete rewrite.

1. What Should Remain Untouched

- **Local Analysis Engine** (src/domains/analysis/engine/): The deterministic SMC/ICT engine with 12-signal confluence scoring is the crown jewel of the platform. It produces reliable, reproducible results without API dependencies. It should be preserved and enhanced, never replaced.
- **Supabase Auth + RLS**: The authentication system and Row Level Security policies are well-implemented. The Telegram HMAC verification is correctly implemented per official documentation. These should remain as the foundation.
- **DDD Domain Structure** (src/domains/): The 9-domain architecture is clean and well-organized. Each domain has clear boundaries with types, functions, and server modules. This structure should be the canonical organization going forward.
- **Design System** (src/styles.css): The Bloomberg Professional dark theme with trading-specific semantic colors is well-designed and should be preserved.
- **Price Fetcher Multi-Source Fallback**: The Binance -> TwelveData -> exchangerate-api -> cache -> null strategy is robust and never fabricates data. This pattern should be the model for all data fetching.

2. What Should Be Refactored

- **fix-vercel-bundle.mjs:** Remove the API route interception stubs. The actual API routes (server/api/) should be registered with Nitro properly so they work without post-build patching. The NFT trace fix can remain, but the API interception and error handler patches should be removed or significantly simplified.
- **Triple Code Duplication:** Consolidate to a single source of truth. Keep src/shared/ as canonical, remove src/lib/ re-exports (update all imports), remove src/integrations/supabase/ (update all imports), remove src/server/ re-exports (update all imports). This is a large but mechanical refactor.
- **Payment System:** Add actual payment verification to purchasePack(), subscribePremium(), and the Telegram webhook. The webhook should validate the X-Telegram-Bot-API-Secret-Token header and verify payment completion before crediting points.
- **Agent Orchestrator:** Replace dynamic imports with static imports. Add rate limiting. Implement streaming for copilot responses. Consider caching agent responses for common queries.
- **Settings Persistence:** Add a user_settings table or JSONB column in profiles to persist user preferences. This is a small but high-impact change.

3. What Should Be Rebuilt

- **Alert System:** The entire alert pipeline needs to be rebuilt: (1) fix the cron endpoint to actually check alerts, (2) add WebSocket-based real-time price monitoring instead of 5-minute polling, (3) add browser push notifications and Telegram notifications, (4) link alerts to analysis results for context.
- **Navigation Architecture:** The current bottom nav + hidden pages pattern does not scale. Consider a sidebar navigation on desktop, a tab-based system with a "More" drawer on mobile, or a command palette (kbar) for power users.
- **Copilot as Tool User:** The copilot should be able to take actions, not just give advice. It should be able to create alerts, add to watchlists, create trades, and fetch real-time data. This requires implementing a tool/function calling architecture on top of the existing agent system.

4. What Should Be Removed

- **vixor-APP/ directory:** Complete project clone, doubles repository size, contains stale migrations.
- **~80 root "(1)" files:** File sync conflict artifacts, not imported by any module.
- **Dead API routes:** src/routes/api/-check-alerts.ts, -generate-signals.ts, -telegram-webhook.ts, -migrate.ts.
- **src/types/api-routes.d.ts:** Dead module declaration for non-existent createAPIFileRoute.
- **src/lib/vixor-mock.ts:** Deprecated mock data module.
- **src/lib/analysis/:** Duplicate of src/domains/analysis/engine/.
- **Dead code in engine.ts:** generateNewsContext() function (170 lines) and calculateConfidence() function.

- **Unused ETF/fundamentals integration:** TwelveData ETF directory, performance, summary, cash flow, earnings estimates, and growth estimates endpoints that are not used by any page.
- **3 stray SQL files at root:** Belong in supabase/migrations/.
- **.env.production with OIDC token:** Remove from git history, rotate the token.

5. What Should Be Postponed

- **Community features:** Social trading, shared watchlists, signal sharing, leaderboards. Valuable but not critical for core trading workflow.
- **Backtesting engine:** Would leverage the existing local analysis engine but requires significant UI and data infrastructure work.
- **Broker API integration:** Auto-import trades from broker accounts. Requires partnerships, compliance review, and significant security work.
- **Advanced order types:** Trailing stops, OCO orders, bracket orders. Requires broker integration.
- **Mobile native app:** The current Telegram Mini App + responsive web approach is sufficient for MVP.

Phased Roadmap

Phase	Focus	Key Actions	Duration	Dependencies
Immediate	Fix Critical Bugs	Fix API stubs in fix-vercel-bundle.mjs; Add payment verification; Remove OIDC token; Make CRON_SECRET required	1-2 weeks	None
Phase 1	Consolidate & Clean	Remove triple duplication; Delete dead code/files; Fix settings persistence; Add Sentry monitoring; Add CI/CD pipeline	2-3 weeks	Immediate fixes
Phase 2	Neural Connections	Analysis-to-Alert flow; Signal-to-Trade flow; Trade-to-Journal flow; Economic Calendar to Daily Loop; Copilot tool use	3-4 weeks	Phase 1 complete
Phase 3	Real-Time & Push	WebSocket price feeds; Real-time alert triggering; Browser push notifications; Streaming copilot responses; Rate limiting	3-4 weeks	Phase 2 complete
Phase 4	Scale & Polish	Navigation redesign; Asset master table; Advanced portfolio analytics; Community features; Backtesting	4-6 weeks	Phase 3 complete